QiskitError: 'Sum of amplitudes-squared does not equal one.' in QEOM wh

https://github.com/qiskit-community/qiskit-nature/issues/373

ROW 84

QiskitError: 'Sum of amplitudes-squared does not equal one.' in QEOM when two qubit reduction is set True #373

New issue

Closed #432

HuangJunye opened on Sep 24, 2021 · edited by HuangJunye Edits

Information

- **Qiskit Nature version**: 0.2.0
- **Python version**: 3.8.8
- **Operating system**: macOS 11.4 Big Sur

What is the current behavior?

Error message: `QiskitError: 'Sum of amplitudes-squared does not equal one.'`

```
---------------------------------------------------------------------------
QiskitError                               Traceback (most recent call last)
<ipython-input-6-a9e19ed18bca> in <module>
      6 vqe_ground_state_solver = GroundStateEigensolver(qubit_converter=converter, solver=algorithm)
      7 qeom_excited_state_solver = QEOM(ground_state_solver=vqe_ground_state_solver)
----> 8 qeom_results = qeom_excited_state_solver.solve(problem=problem)
      9
     10 ######## Write your code between these lines

/usr/local/anaconda3/envs/oled-new/lib/python3.8/site-packages/qiskit_nature/algorithms/excited_states_solvers
    110
    111         # 3. Evaluate eom operators
--> 112         measurement_results = self._gsc.evaluate_operators(
    113             groundstate_result.eigenstates[0], matrix_operators_dict
    114         )

/usr/local/anaconda3/envs/oled-new/lib/python3.8/site-packages/qiskit_nature/algorithms/ground_state_solvers/g
    164                     results[name] = None
    165                 else:
--> 166                     results[name] = self._eval_op(state, op, quantum_instance, expectation)
    167         else:
    168             if operators is None:

/usr/local/anaconda3/envs/oled-new/lib/python3.8/site-packages/qiskit_nature/algorithms/ground_state_solvers/g
    187                 if expectation is not None:
    188                     exp = expectation.convert(exp)
--> 189                 result = sampler.convert(exp).eval()
    190             except ValueError:
    191                 # TODO make this cleaner. The reason for it being here is that some quantum

/usr/local/anaconda3/envs/oled-new/lib/python3.8/site-packages/qiskit/opflow/converters/circuit_sampler.py in
    174
    175             # convert to circuit and reduce
--> 176             operator_dicts_replaced = operator.to_circuit_op()
    177             self._reduced_op_cache = operator_dicts_replaced.reduce()
    178

/usr/local/anaconda3/envs/oled-new/lib/python3.8/site-packages/qiskit/opflow/list_ops/list_op.py in to_circuit
    558             ).reduce()
    559         return self.__class__(
--> 560             [
    561                 op.to_circuit_op() if not isinstance(op, OperatorStateFn) else op
    562                 for op in self.oplist

/usr/local/anaconda3/envs/oled-new/lib/python3.8/site-packages/qiskit/opflow/list_ops/list_op.py in <listcomp>
    559         return self.__class__(
    560             [
--> 561                 op.to_circuit_op() if not isinstance(op, OperatorStateFn) else op
    562                 for op in self.oplist
    563             ],

/usr/local/anaconda3/envs/oled-new/lib/python3.8/site-packages/qiskit/opflow/state_fns/vector_state_fn.py in t
    162         from .circuit_state_fn import CircuitStateFn
    163
--> 164         csfn = CircuitStateFn.from_vector(self.primitive.data) * self.coeff
    165         return csfn.adjoint() if self.is_measurement else csfn
    166

/usr/local/anaconda3/envs/oled-new/lib/python3.8/site-packages/qiskit/opflow/state_fns/circuit_state_fn.py in
    124         normalization_coeff = np.linalg.norm(statevector)
    125         normalized_sv = statevector / normalization_coeff
--> 126         return CircuitStateFn(Initialize(normalized_sv), coeff=normalization_coeff)
```

Assignees

mrossinek

woodsp-ibm

Labels

status: confirmed type: bug

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

Fix EOM with 2Q reduction and tapering
qiskit-community/qiskit-nature

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

```
--> 126         return CircuitStateFn(Initialize(normalized_sv), coeff=normalization_coeff)
    127
    128     def primitive_strings(self) -> Set[str]:

/usr/local/anaconda3/envs/oled-new/lib/python3.8/site-packages/qiskit/extensions/quantum_initializer/initializ
     90             # Check if probabilities (amplitudes squared) sum to 1
     91             if not math.isclose(sum(np.absolute(params) ** 2), 1.0, abs_tol=_EPS):
---> 92                 raise QiskitError("Sum of amplitudes-squared does not equal one.")
     93
     94             num_qubits = int(num_qubits)

QiskitError: 'Sum of amplitudes-squared does not equal one.'
```

Steps to reproduce the problem

Add these code after the example code in the README.

```
from qiskit_nature.algorithms import QEOM
from qiskit_nature.algorithms import GroundStateEigensolver

vqe_ground_state_solver = GroundStateEigensolver(qubit_converter=converter, solver=algorithm)
qeom_excited_state_solver = QEOM(ground_state_solver=vqe_ground_state_solver)
qeom_results = qeom_excited_state_solver.solve(problem=problem)

print(qeom_results)
```

What is the expected behavior?

Returns ElectronicStructureResult. This is the result when `two_qubit_reduction` is set to `False` in `QubitConverter`:

```
=== GROUND STATE ENERGY ===

* Electronic ground state energy (Hartree): -1.857107039335
  - computed part:      -1.857107039335
~ Nuclear repulsion energy (Hartree): 0.719968994449
> Total ground state energy (Hartree): -1.137138044886

=== EXCITED STATE ENERGIES ===

  1:
* Electronic excited state energy (Hartree): -1.244589346215
> Total excited state energy (Hartree): -0.524620351766
  2:
* Electronic excited state energy (Hartree): -0.882883155693
> Total excited state energy (Hartree): -0.162914161244
  3:
* Electronic excited state energy (Hartree): -0.225098418795
> Total excited state energy (Hartree): 0.494870575654

=== MEASURED OBSERVABLES ===

  0:  # Particles: 2.000 S: 0.000 S^2: 0.000 M: 0.000

=== DIPOLE MOMENTS ===

~ Nuclear dipole moment (a.u.): [0.0  0.0  1.3889487]

  0:
  * Electronic dipole moment (a.u.): [0.0  0.0  1.40580143]
    - computed part:      [0.0  0.0  1.40580143]
  > Dipole moment (a.u.): [0.0  0.0  -0.01685273]  Total: 0.01685273
                 (debye): [0.0  0.0  -0.04283535]  Total: 0.04283535
```

Suggested solutions

HuangJunye on Sep 24, 2021 — Contributor — Author

Here's the gist of a notebook reproducing the error:
https://gist.github.com/HuangJunye/cce1a2c2f6a04cc865eeb54170288acb

mrossinek added status: confirmed type: bug on Sep 29, 2021

mrossinek on Sep 29, 2021 Member

This issue boils down to a mismatching qubit operator of the ground state Hamiltonian and the hopping operators. All of these are checked for their commutativity. QEOM appears to rely on using the *untapered* qubit op here. However, the hopping operators are actually already being mapped+reduced+tapered. This causes a mismatch.

The question is now whether the reliance on using the untapered operator is actually required or whether using `convert_match` instead of `map` on the line linked above is sufficient. It appears to fix this particular bug but it will still cause problems when using `z2symmetry_reduction="auto"` (in which case a straight forward test appears to run into the same issue as #342).

If QEOM needs to perform tapering manually then the internal Z2Symmetry which is being used inside of `TwoQubitReduction` needs to be made available in `QubitConverter.z2symmetries` (which is where QEOM gets the symmetry information from).

mrossinek on Sep 29, 2021 · edited by mrossinek Edits Member

Here is a "partial fix" which changes `map -> convert_match` and adds two unittests for QEOM:

- one with two-qubit reduction (which works with this applied patch)
- one with automatic `Z2Symmetries` (which runs into [EDIT: here used to be a linked issue. But it was the wrong ID and I don't recall which one was supposed to be linked to.])

```
diff --git a/qiskit_nature/algorithms/excited_states_solvers/qeom.py b/qiskit_nature/algorithms/excited_
index d0987c189..8b98c0252 100644
--- a/qiskit_nature/algorithms/excited_states_solvers/qeom.py
+++ b/qiskit_nature/algorithms/excited_states_solvers/qeom.py
@@ -105,7 +105,7 @@ class QEOM(ExcitedStatesSolver):
         groundstate_result = self._gsc.solve(problem)

         # 2. Prepare the excitation operators
-        self._untapered_qubit_op_main = self._gsc._qubit_converter.map(problem.second_q_ops()[0])
+        self._untapered_qubit_op_main = self._gsc._qubit_converter.convert_match(problem.second_q_ops()[0])
         matrix_operators_dict, size = self._prepare_matrix_operators(problem)

         # 3. Evaluate eom operators
diff --git a/test/algorithms/excited_state_solvers/test_excited_states_solvers.py b/test/algorithms/excited_st
index f1eac8f7e..34c5e859b 100644
--- a/test/algorithms/excited_state_solvers/test_excited_states_solvers.py
+++ b/test/algorithms/excited_state_solvers/test_excited_states_solvers.py
@@ -21,7 +21,7 @@ from qiskit.algorithms import NumPyMinimumEigensolver, NumPyEigensolver

 from qiskit_nature.drivers import UnitsType
 from qiskit_nature.drivers.second_quantization import PySCFDriver
-from qiskit_nature.mappers.second_quantization import JordanWignerMapper
+from qiskit_nature.mappers.second_quantization import JordanWignerMapper, ParityMapper
 from qiskit_nature.converters.second_quantization import QubitConverter
 from qiskit_nature.problems.second_quantization import ElectronicStructureProblem
 from qiskit_nature.algorithms import (
@@ -85,6 +85,28 @@ class TestNumericalQEOMESCCalculation(QiskitNatureTestCase):
         for idx, energy in enumerate(self.reference_energies):
             self.assertAlmostEqual(results.computed_energies[idx], energy, places=4)

+    def test_vqe_mes_2q(self):
+        """Test VQEUCCSDFactory with QEOM and 2-qubit reduction"""
+        converter = QubitConverter(ParityMapper(), two_qubit_reduction=True)
+        solver = VQEUCCFactory(self.quantum_instance)
+        gsc = GroundStateEigensolver(converter, solver)
+        esc = QEOM(gsc, "sd")
+        results = esc.solve(self.electronic_structure_problem)
+
+        for idx, energy in enumerate(self.reference_energies):
+            self.assertAlmostEqual(results.computed_energies[idx], energy, places=4)
+
```

```
@@ -105,7 +105,7 @@ class QEOM(ExcitedStatesSolver):
         groundstate_result = self._gsc.solve(problem)

         # 2. Prepare the excitation operators
-        self._untapered_qubit_op_main = self._gsc._qubit_converter.map(problem.second_q_ops()[0])
+        self._untapered_qubit_op_main = self._gsc._qubit_converter.convert_match(problem.second_q_ops()[0])
         matrix_operators_dict, size = self._prepare_matrix_operators(problem)

         # 3. Evaluate eom operators
diff --git a/test/algorithms/excited_state_solvers/test_excited_states_solvers.py b/test/algorithms/excited_st
index f1eac8f7e..34c5e859b 100644
--- a/test/algorithms/excited_state_solvers/test_excited_states_solvers.py
+++ b/test/algorithms/excited_state_solvers/test_excited_states_solvers.py
@@ -21,7 +21,7 @@ from qiskit.algorithms import NumPyMinimumEigensolver, NumPyEigensolver

 from qiskit_nature.drivers import UnitsType
 from qiskit_nature.drivers.second_quantization import PySCFDriver
-from qiskit_nature.mappers.second_quantization import JordanWignerMapper
+from qiskit_nature.mappers.second_quantization import JordanWignerMapper, ParityMapper
 from qiskit_nature.converters.second_quantization import QubitConverter
 from qiskit_nature.problems.second_quantization import ElectronicStructureProblem
 from qiskit_nature.algorithms import (
@@ -85,6 +85,28 @@ class TestNumericalQEOMESCCalculation(QiskitNatureTestCase):
         for idx, energy in enumerate(self.reference_energies):
             self.assertAlmostEqual(results.computed_energies[idx], energy, places=4)

+    def test_vqe_mes_2q(self):
+        """Test VQEUCCSDFactory with QEOM and 2-qubit reduction"""
+        converter = QubitConverter(ParityMapper(), two_qubit_reduction=True)
+        solver = VQEUCCFactory(self.quantum_instance)
+        gsc = GroundStateEigensolver(converter, solver)
+        esc = QEOM(gsc, "sd")
+        results = esc.solve(self.electronic_structure_problem)
+
+        for idx, energy in enumerate(self.reference_energies):
+            self.assertAlmostEqual(results.computed_energies[idx], energy, places=4)
+
+    def test_vqe_mes_z2(self):
+        """Test VQEUCCSDFactory with QEOM and Z2Symmetries"""
+        converter = QubitConverter(ParityMapper(), two_qubit_reduction=True, z2symmetry_reduction="auto")
+        solver = VQEUCCFactory(self.quantum_instance)
+        gsc = GroundStateEigensolver(converter, solver)
+        esc = QEOM(gsc, "sd")
+        results = esc.solve(self.electronic_structure_problem)
+
+        for idx, energy in enumerate(self.reference_energies):
+            self.assertAlmostEqual(results.computed_energies[idx], energy, places=4)
+
     def test_numpy_factory(self):
         """Test NumPyEigenSolverFactory with ExcitedStatesEigensolver"""
```

mrossinek assigned mrossinek and woodsp-ibm on Sep 29, 2021

woodsp-ibm mentioned this on Nov 16, 2021

Fix EOM with 2Q reduction and tapering #432

mrossinek closed this as completed in #432 on Nov 22, 2021

Fix EOM with 2Q reduction and tapering #432

Code

Merged mrossinek merged 7 commits into `qiskit-community:main` from `woodsp-ibm:qeom_fix` on Nov 22, 2021

Conversation 6 · Commits 7 · Checks 0 · Files changed 5 · +70 −10

woodsp-ibm commented on Nov 16, 2021 • edited · Member

Change so that operator, and hopping operators, are just mapped and possibly 2Q reduced (i.e. no general Z2Symmetry tapering). This matches the QEOM logic from before in Aqua. Fixed the (anti)-commutation logic and changed so it no longer uses deprecated logic (this logic was not covered by the unit tests before at all and was missed when other use of deprecated code was taken care of recently).

Fixes #373

I have added some unit tests - I chose to do it this way, rather than using ddt, as its easier to run one individually.

- [x] Add release note
- [x] Sort out remaining failure when Parity mapping is used without 2Q reduction but with Z2 symmetries. In looking at/fixing QiskitError: 'Sum of amplitudes-squared does not equal one.' in QEOM when two qubit reduction is set True #373 I noticed, when trying out various configurations to check the ground stat,e that this same configuration, with the molecule in that code sample from that issue, would give an incorrect result. It did not fail but it seemed to be choosing a wrong sector (exact diagonalization via NumPy algo gave same answer so it was not a failure of VQE, as used in that sample, to converge). I am not sure if the failure here when using UCCSD, is the exact same issue; but it's the same conversion config so it seems related.

Update: The above was indeed choosing the wrong sector. The sector selector based on HF bitstring was unchanged from the implementation in Aqua chemistry. However the HF bitstring was changed, when the code was refactored/moved to Nature, so that things are all little endian - and as such the HF bitstring was reversed. The pick sector in Aqua chemistry had the symettry terms reversed sym.z[::-1] which would have aligned to the way it was needed in Aqua, since the HF string is now reversed in Nature I removed the sym.z reversal and the sector now selects the correct one.

2

`Fix EOM with 2Q reduction and tapering` fb83d34

woodsp-ibm requested review from **manoelmarques**, **mrossinek**, **pbark** and **stefan-woerner** as code owners 4 years ago

woodsp-ibm and others added 2 commits 4 years ago

`Merge branch 'main' into qeom_fix` Verified 479912e

`Fix sector picker` c37655d

coveralls commented on Nov 19, 2021 • edited

Pull Request Test Coverage Report for Build 1490536673

- **2** of **7** **(28.57%)** changed or added relevant lines in **3** files are covered.
- No unchanged relevant lines lost coverage.
- Overall coverage increased **(+0.07%)** to **85.63%**

Reviewers
- mrossinek ✓
- manoelmarques
- pbark
- stefan-woerner

Assignees
No one assigned

Labels
None yet

Projects
None yet

Milestone
No milestone

Development
Successfully merging this pull request may close these issues.
QiskitError: 'Sum of amplitudes-squared doe...

Notifications Customize
Subscribe
You're not receiving notifications from this thread.

3 participants